PENETRATION TESTING PROJECT PLAN

Next-Gen AI-Powered WAF
& Real-Time SOC Telemetry Platform

Four-Week Design, Development & Reporting Plan

Project Duration: October 12 – November 10
Team Size: 5 Members

CONFIDENTIAL — Academic / Lab Use Only

1. Executive Summary

Our team builds a Next-Generation Web Application Firewall (WAF) that protects an intentionally vulnerable web application in an isolated lab. The WAF combines fast signature-based inspection, input de-obfuscation and a lightweight machine-learning classifier to detect attacks and suspicious encoded requests. It also provides rate limiting, event logging, threat intelligence, Telegram alerts and a real-time SOC dashboard. After finishing the build, we test the WAF with authorized attack tools, measure the results, and write a clear report.

1.1 Scope & Rules of Engagement

All testing runs only against our own Docker containers or authorized local lab targets. We do not touch any public or third-party system. External threat-intelligence queries are limited to lab metadata, and API credentials are stored securely.

2. Technologies & Tools

Category	Tools
Application & Runtime	Python, FastAPI, httpx, Docker, OWASP Juice Shop
Detection & Normalization	Python re, urllib, base64; structured signature rules
AI / ML	Pandas, Scikit-learn, TF-IDF, CSIC 2010 or labeled HTTP dataset
Data & Telemetry	SQLite, Streamlit, Plotly
Response & Intelligence	Rate limiting, AbuseIPDB API, Telegram Bot API
Security Testing	Burp Suite, sqlmap, Nikto
Documentation	GitHub, Markdown, MS Word

3. Timeline & Weekly Tasks (4-Week Plan)

Week 1 — Foundation & Traffic Flow *(Oct 12 – Oct 19)*

- Set up GitHub workflow, Docker target application and isolated testing environment.
- Implement a working FastAPI/httpx reverse proxy and HTTP request capture.
- Define event schema, core module interfaces and basic URL/case normalization.

Deliverable: *Running lab, proxy.py, initial normalizer.py and architecture document.*

Week 2 — Dual Detection Engines *(Oct 20 – Oct 26)*

- Build signatures for SQLi, XSS, path traversal and command injection.
- Prepare labeled HTTP data and train a TF-IDF classifier; evaluate precision, recall and F1.
- Integrate signature verdicts and ML predictions in detector.py.

Deliverable: *Signature rules, vectorizer.pkl, model.pkl, detector.py and ML evaluation report.*

Week 3 — Response & SOC Telemetry *(Oct 27 – Nov 2)*

- Implement rate limiting with clear thresholds and temporary blocking.
- Save security events to SQLite and display dashboard trends.
- Integrate Telegram alerts and AbuseIPDB, with secrets stored in environment variables.
- Run end-to-end tests for allow, block, alert and logging.

Deliverable: *Working dashboard, threat intelligence, Telegram alerts and integrated demo.*

Week 4 — Final Testing, Report & Delivery *(Nov 3 – Nov 10)*

- Test the WAF with authorized Burp Suite, sqlmap and Nikto lab cases.
- Measure detection rate, false positives, added latency and alert time.
- Fix defects, document limitations and update the GitHub repository.
- Write the final report and prepare the presentation and live demo.

Deliverable: *Complete report, benchmark evidence, presentation and runnable GitHub repository.*

4. Team Roles (5 Members)

Member	Primary Role	Assigned Tasks
Ali Soliman (Team Lead)	Active Defense & Pentesting	Rate limiting, AbuseIPDB and Telegram connectors, Burp/sqlmap/Nikto testing, benchmark evidence, and managing the GitHub repository.
Mohamed Magdy	Reverse Proxy & Integration	Building proxy.py, request routing, blocking enforcement, and the Docker environment.
Hesham	AppSec Rules & Normalization	Writing normalizer.py, attack signatures, and unit tests for SQLi, XSS, traversal and injection.
Ahmed Magdy	SOC Dashboard & Logging	Designing the SQLite events database, building dashboard.py, and taking SOC screenshots.
Mohamed Abdulrahman Mohamed	AI/ML Security	Cleaning the dataset, training the TF-IDF model, and reporting metrics and confusion matrix.